



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DSA0501 - Query Processing using Data Analysis

Name: Rakshitha V B

Reg.No: 192324004

Dept: B.Tech AI&DS

1. Write a Pandas program to select the distinct department id from employee's file.

Aim:

To write a Python program using Pandas to display the unique department IDs from the departments dataset.

Algorithm

1. Import the Pandas library.
2. Read the departments.csv file.
3. Select the DEPARTMENT_ID column.
4. Use the unique() function to get distinct department IDs.
5. Display the result.

Code:

```
import pandas as pd
data = pd.read_csv('Csv/Lab1.csv')
distinct_id = data['DEPARTMENT_ID'].unique()
print("Distinct Department IDs:")
print(distinct_id)
```

Output:

```
Distinct Department IDs:
[ 10  20  30  40  50  60  70  80  90 100 110 120 130 140 150 160 170 180
 190 200 210 220 230 240 250 260 270]
```

Result:

Thus, the required operation was performed successfully using the Pandas library.

2. Write a Pandas program to display the IDs of employees who have done two or more jobs in the past.

Aim:

To write a Python program using Pandas to find employees who have worked in two or more jobs. Algorithm

1. Import the Pandas library.
2. Read the job_history.csv file.
3. Group the data by EMPLOYEE_ID.
4. Count the number of job records for each employee.
5. Display employee IDs having two or more job records.

Code:

```
import pandas as pd

data = pd.read_csv('Csv/Lab2.csv')

result = data.groupby("EMPLOYEE_ID").filter(lambda x: len(x) >= 2 ).
sort_values (by="EMPLOYEE_ID")

print("Employees who have done two or more jobs:")

print(result)
```

Output:

Employees who have done two or more jobs:					
	EMPLOYEE_ID	START_DATE	END_DATE	JOB_ID	DEPARTMENT_ID
1	101	1997-09-21	2001-10-27	AC_ACCOUNT	110
2	101	2001-10-28	2005-03-15	AC_MGR	110
7	176	2006-03-24	2006-12-31	SA_REP	80
8	176	2007-01-01	2007-12-31	SA_MAN	80
6	200	1995-09-17	2001-06-17	AD_ASST	90
9	200	2002-07-01	2006-12-31	AC_ACCOUNT	90

Result:

Thus, the required operation was performed successfully using the Pandas library. The desired records were displayed correctly based on the given condition.

3. Write a Pandas program to display the details of jobs in descending sequence on job title.

Aim:

To write a Python program using Pandas to sort the jobs dataset in descending order of job title. Algorithm

1. Import the Pandas library.
2. Read the jobs.csv file.
3. Sort the data using the JOB_TITLE column in descending order.
4. Display the sorted data.

Code:

```
import pandas as pd  
data = pd.read_csv('Csv/Lab3.csv')  
result = data.sort_values(by="JOB_TITLE", ascending=False)  
print(result)
```

Output:

	JOB_ID	JOB_TITLE	MIN_SALARY	MAX_SALARY
11	ST_MAN	Stock Manager	5500	8500
12	ST_CLERK	Stock Clerk	2008	5000
13	SH_CLERK	Shipping Clerk	2500	5500
8	SA_REP	Sales Representative	6000	12008
7	SA_MAN	Sales Manager	10000	20080
9	PU_MAN	Purchasing Manager	8000	15000
10	PU_CLERK	Purchasing Clerk	2500	5500
18	PR_REP	Public Relations Representative	4500	10500
6	AC_ACCOUNT	Public Accountant	4200	9000
14	IT_PROG	Programmer	4000	10000
0	AD_PRES	President	20080	40000
16	MK_REP	Marketing Representative	4000	9000
15	MK_MAN	Marketing Manager	9000	15000
17	HR_REP	Human Resources Representative	4000	9000
3	FI_MGR	Finance Manager	8200	16000
1	AD_VP	Administration Vice President	15000	30000
2	AD_ASST	Administration Assistant	3000	6000
5	AC_MGR	Accounting Manager	8200	16000
4	FI_ACCOUNT	Accountant	4200	9000

Result:

Thus, the required operation was performed successfully using the Pandas library. The desired records were displayed correctly based on the given condition.

4. Write a Pandas program to create a line plot of the historical stock prices of Alphabet Inc. between two specific dates.

Aim:

To create a line plot of the historical stock prices of Alphabet Inc. between two specific dates using Pandas and Matplotlib.

Algorithm

1. Import the required libraries (pandas and matplotlib.pyplot).
2. Read the stock price dataset (alphabet_stock.csv) using pd.read_csv().
3. Convert the Date column into datetime format using pd.to_datetime().
4. Specify the start date and end date for the required period.
5. Filter the dataset to include only the records between the selected dates.
6. Plot the Closing Price against the Date using plt.plot().
7. Add the chart title, x-axis label, y-axis label, and grid.
8. Display the line plot using plt.show().

Code:

```
import pandas as pd
import matplotlib.pyplot as plt # type: ignore

df = pd.read_csv("Csv/Lab4.csv")

df["Date"] = pd.to_datetime(df["Date"])
start_date = "2022-01-05"
end_date = "2022-01-12"

stock = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]

plt.figure(figsize=(8,5))
```

```
plt.plot(stock["Date"], stock["Close"], marker="o")
```

```
plt.title("Historical Stock Prices of Alphabet Inc.")
```

```
plt.xlabel("Date")
```

```
plt.ylabel("Closing Price ($)")
```

```
plt.grid(True)
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```

Output:



Result:

The historical stock prices of Alphabet Inc. between the specified dates were successfully filtered and displayed as a line graph using Pandas and Matplotlib. The graph clearly shows the variation in the company's closing stock prices over the selected time period.

5. Write a Pandas program to create a bar plot of the trading volume of Alphabet Inc. stock between two specific dates.

Aim:

To create a bar plot of the trading volume of Alphabet Inc. stock between two specific dates using Pandas and Matplotlib.

Algorithm

1. Import the required libraries (pandas and matplotlib.pyplot).
2. Read the stock dataset (alphabet_stock.csv) using pd.read_csv().
3. Convert the Date column into datetime format using pd.to_datetime().
4. Specify the start date and end date.
5. Filter the records between the selected dates.
6. Plot the Volume against the Date using plt.bar().
7. Add the chart title, axis labels, and rotate the date labels.
8. Display the bar plot using plt.show()

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Csv/Lab4.csv")

df["Date"] = pd.to_datetime(df["Date"])

start_date = "2022-01-05"
end_date = "2022-01-12"

stock = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]

plt.figure(figsize=(8,5))
```

```
plt.bar(stock["Date"].dt.strftime("%Y-%m-%d"), stock["Volume"])
```

```
plt.title("Trading Volume of Alphabet Inc.")
```

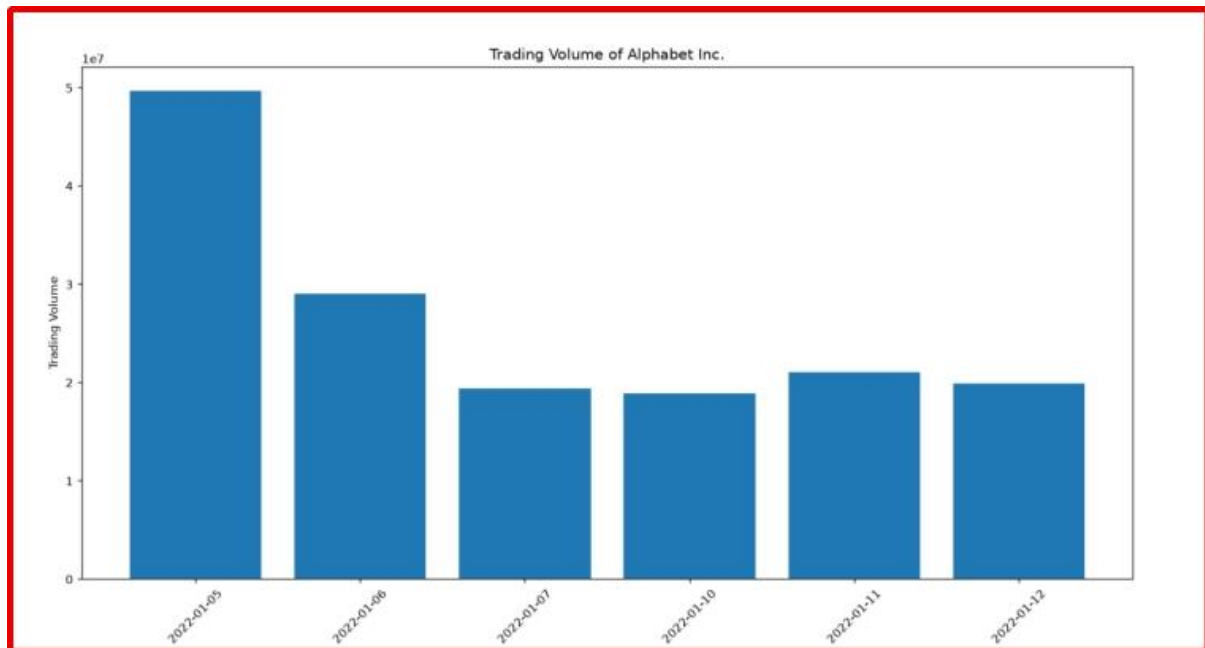
```
plt.xlabel("Date")
```

```
plt.ylabel("Trading Volume")
```

```
plt.xticks(rotation=45)
```

```
plt.show()
```

Output:



Result:

The trading volume of Alphabet Inc. stock between the specified dates was successfully filtered and displayed as a bar plot using Pandas and Matplotlib. The chart clearly represents the daily trading volume for the selected period.

6. Write a Pandas program to create a scatter plot of the trading volume prices of Alphabet Inc. stock between two specific dates.

Aim:

To create a scatter plot of the trading volume and closing stock prices of Alphabet Inc. between two specific dates using Pandas and Matplotlib.

Algorithm

1. Import the required libraries (pandas and matplotlib.pyplot).
2. Read the dataset alphabet_stock_data.csv using pd.read_csv().
3. Convert the **Date** column into datetime format.
4. Specify the start date and end date.
5. Filter the dataset between the specified dates.
6. Create a scatter plot with **Volume** on the x-axis and **Close** (stock price) on the y-axis.
7. Add chart title, axis labels, and grid.
8. Display the scatter plot.

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv("Csv/Lab6.csv")

df["Date"] = pd.to_datetime(df["Date"])

start_date = "2020-04-01"
end_date = "2020-05-01"

stock = df[(df["Date"] >= start_date) & (df["Date"] <= end_date)]
```

```
plt.figure(figsize=(8,5))
plt.scatter(stock["Volume"], stock["Close"])

plt.title("Trading Volume vs Stock Price of Alphabet Inc.")
plt.xlabel("Trading Volume")
plt.ylabel("Closing Stock Price")
plt.grid(True)

plt.show()
```

Output:



Result:

The scatter plot of trading volume versus closing stock prices of Alphabet Inc. between the specified dates was successfully created using Pandas and Matplotlib. The plot illustrates the relationship between the trading volume and the stock's closing price over the selected period.

7. Write a Pandas program to create a Pivot table and find the maximum and minimum sale value of the items.

Aim:

To create a Pivot Table and find the maximum and minimum sale value of each item using Pandas.

Algorithm

1. Import the Pandas library.
2. Read the sales_data.csv file using pd.read_csv().
3. Create a pivot table using pd.pivot_table().
4. Set Item as the index. Use Sale_amt as the values.
5. Apply the aggregation functions max and min. Display the pivot table.

Code:

```
import pandas as pd
df = pd.read_csv("Csv/Lab7.csv")
pivot = pd.pivot_table(
    df, index="Item", values="Sale_amt", aggfunc=["max", "min"]
)
print(pivot)
```

Output:

	max	min
	Sale_amt	Sale_amt
Item		
Cell Phone	14400	6075
Desk	250	250
Home Theater	40500	14000
Television	113810	38336
Video Games	936	936

Result:

The pivot table was successfully created, displaying the maximum and minimum sale values for each item in the sales dataset using Pandas.

8. Write a Pandas program to create a Pivot table and find the item wise unit sold.

Aim:

To create a Pivot Table and find the item-wise total units sold using the Pandas library.

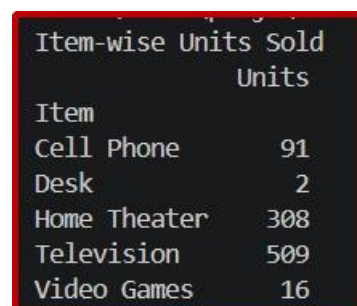
Algorithm

1. Import the Pandas library.
2. Read the sales_data.csv file.
3. Create a pivot table using pd.pivot_table().
4. Set Item as the index.
5. Use Units as the values.
6. Apply the sum aggregation function.
7. Display the pivot table.

Code:

```
import pandas as pd
df = pd.read_csv("Csv/Lab7.csv")
pivot = pd.pivot_table(
    df, index="Item", values="Units", aggfunc="sum")
print("Item-wise Units Sold")
print(pivot)
```

Output:



Item-wise Units Sold	
Item	Units
Cell Phone	91
Desk	2
Home Theater	308
Television	509
Video Games	16

Result:

The Pivot Table was successfully created, and the item-wise total units sold were calculated and displayed using the Pandas library.